

Informe Desarrollo Móvil I (SCAM)



Versiones del documento

Fecha	Versión	Descripción	Autor
Junio	1.0	Versión inicial del sistema SCAM con marcado de asistencia, panel admin, GPS y control de dispositivos	Nestor Garcia, Jason Madrid, Gabriel Jimenez, Dalvin Soriano



Contenido

Informe Desarrollo Móvil I	4
1. DESCRIPCIÓN DE LA SOLUCIÓN	4
1.1 Resumen gerencial	4
1.2 Glosario	5
1.3 Restricciones y metas de arquitectura	6
1.4 Diagrama de Proceso.....	7
1.5 Requerimientos del sistema	9
1.5.1 Vista de Casos de uso de la aplicación	10
2. Tecnología 11	
2.1 Desarrollo	11
2.2 Modelo Entidad Relación.....	13

Informe Desarrollo Móvil I

1. DESCRIPCIÓN DE LA SOLUCIÓN

1.1 Resumen gerencial

SCAM (Sistema de Control de Asistencia Móvil) es una solución tecnológica integral diseñada para optimizar y modernizar la gestión y el control de asistencia del capital humano en cualquier empresa u organización. Desarrollada como un proyecto de innovación universitaria para el curso de Desarrollo Móvil I en CEUTEC Honduras, la plataforma destaca por su enfoque en la movilidad, la seguridad de los datos y la validación en tiempo real.

El sistema resuelve de manera eficiente la necesidad organizacional de supervisar la jornada laboral del personal mediante un mecanismo automatizado que mitiga el fraude y mejora la precisión operativa. Sus principales pilares funcionales incluyen:

Autenticación de Usuarios: El personal accede y registra sus marcajes de entrada y salida de forma ágil utilizando su DNI y un PIN de seguridad de cuatro dígitos.

Geofencing (Validación GPS): La aplicación restringe los marcajes, permitiendo que se realicen exclusivamente dentro de un perímetro geográfico predefinido y autorizado por la organización.

Seguridad y Vinculación de Dispositivos: Para garantizar la integridad del proceso, el sistema registra especificaciones técnicas del teléfono móvil del empleado (sistema operativo, marca y modelo), validando en tiempo real que el marcaje se realice desde el dispositivo autorizado.

Autogestión del Empleado: Los usuarios tienen acceso a su historial de asistencia, así como a la actualización de sus datos personales y PIN de acceso.

Panel Administrativo: El administrador cuenta con herramientas centralizadas para la gestión de usuarios y roles, configuración del perímetro GPS, visualización de alertas automáticas y generación de reportes detallados de horas laboradas y horas extras.

Para asegurar un rendimiento óptimo, escalabilidad y alta seguridad, el sistema se construyó bajo el siguiente ecosistema tecnológico:

Frontend (Aplicación Móvil): Desarrollada en React Native con Expo, garantizando una experiencia de usuario fluida y compatibilidad multiplataforma.

Backend y API: Diseñado con Node.js, Express y TypeScript, estructurado a través de una API REST.

Base de Datos: Relacional, implementada en MySQL para un manejo sólido del historial y registros.

Seguridad de Datos: Comunicación protegida mediante tokens de autenticación JWT (JSON Web Tokens).

1.2 Glosario

Término	Definición
DNI	Documento Nacional de Identidad. Identificador único de 13 dígitos de cada empleado hondureño. Es el dato que el empleado escribe en la app para identificarse antes de marcar asistencia.
PIN	Número de 4 dígitos personal de cada empleado. Se cifra con bcryptjs (hash de 60 caracteres) antes de guardarse en la base de datos; nunca se almacena el valor real.
JWT	JSON Web Token. Token digital firmado que el servidor entrega al administrador al iniciar sesión correctamente. Se envía en la cabecera Authorization de cada petición protegida (panel admin) para verificar la identidad sin consultar la base de datos en cada llamada.
GPS	Sistema de Posicionamiento Global. La app captura las coordenadas de latitud y longitud del empleado al marcar mediante expo-location, y el servidor calcula si está dentro del perímetro autorizado.
Device ID	Identificador único del dispositivo móvil, capturado con expo-device (nombre del modelo) y expo-application (Android ID). Se usa para detectar suplantación de identidad: si un empleado intenta marcar desde un celular no registrado, el sistema genera una alerta automática.
Haversine	Fórmula matemática para calcular la distancia real en metros entre dos puntos geográficos sobre la superficie esférica de la Tierra, usando las diferencias de latitud y longitud. El backend la usa para determinar si el empleado está dentro del perímetro autorizado al momento de marcar.
Trigger	Procedimiento almacenado en MySQL que se ejecuta automáticamente después de cada INSERT en la tabla asistencias. Calcula las horas laboradas y horas extras usando TIMESTAMPDIF, y genera alertas (GPS fuera de perímetro, dispositivo no registrado, dispositivo compartido) sin intervención del código del backend.
Perímetro GPS	Área circular definida por un punto central (latitud y longitud configurables) y un radio en metros (por defecto 200 m). Los empleados deben estar dentro de este círculo al marcar asistencia. El administrador puede ajustar las coordenadas y el radio desde el panel de configuración GPS.
Horas extras	Horas trabajadas que superan la jornada estándar configurada por empleado (por defecto 8 horas). Se calculan automáticamente con la fórmula: si horas_laboradas > horas_jornada, entonces

	horas_extras = horas_laboradas - horas_jornada; de lo contrario, horas_extras = 0.
API REST	Interfaz de programación que permite la comunicación entre la app móvil (frontend) y el servidor (backend) mediante peticiones HTTP (GET, POST, PUT) en formato JSON. Todas las rutas del sistema empiezan con /api.
bcryptjs	Librería para el cifrado seguro de contraseñas y PINs mediante el algoritmo bcrypt. Genera un hash unidireccional de 60 caracteres; es imposible obtener el valor original a partir del hash. La verificación se hace comparando el texto plano con el hash usando bcrypt.compare().
Sequelize	ORM (Object-Relational Mapper) para Node.js que permite interactuar con la base de datos MySQL mediante modelos de TypeScript (clases y objetos), sin necesidad de escribir SQL manualmente en cada consulta del API.

1.3 Restricciones y metas de arquitectura

Restricciones del sistema:

- Red local requerida: el dispositivo móvil y el servidor backend deben estar conectados a la misma red WiFi. El backend corre en un servidor local (puerto 5000) sin exposición a internet en la versión 1.0.
- Permisos de ubicación: la validación GPS requiere que el empleado conceda permiso de ubicación al dispositivo. Si lo deniega, la marcada se registra con dentro_perimetro = NULL y no se puede verificar el perímetro.
- Un dispositivo por empleado: el sistema permite registrar únicamente un dispositivo activo por empleado. Si cambia de celular, el administrador debe autorizar el nuevo desde la pantalla de Dispositivos.
- Plataforma Android: la versión 1.0 está diseñada y probada para Android (APK). La versión iOS no forma parte del alcance inicial.
- IP dinámica: la dirección IP del servidor cambia según la red WiFi donde esté conectado. Debe actualizarse en Services/api.ts antes de cada sesión de uso ejecutando ipconfig y copiando la IPv4 actual.

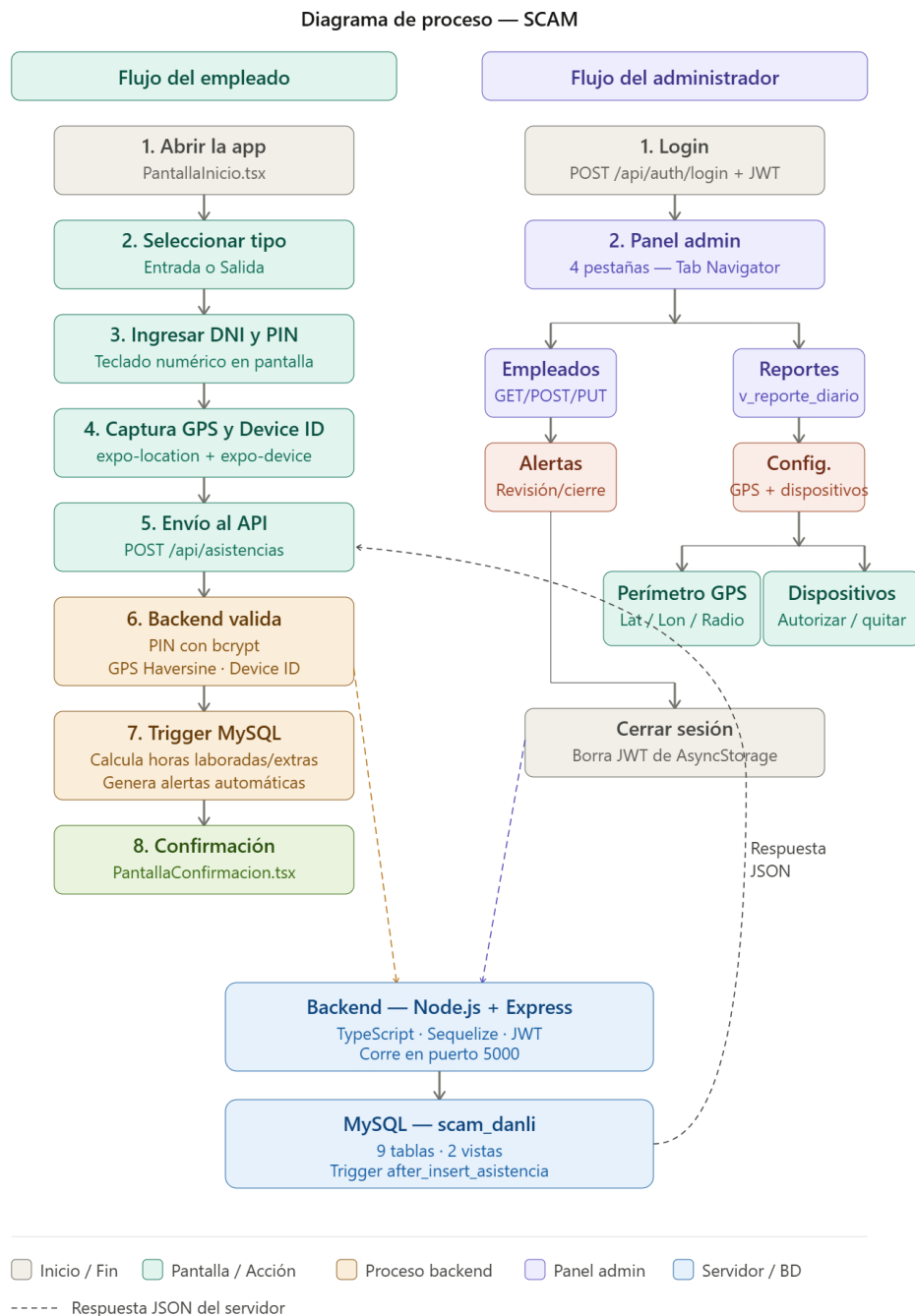
Metas de arquitectura:

- Seguridad: todos los PINs de empleados y contraseñas de administradores se cifran con bcryptjs antes de guardarse. La comunicación con el panel admin está protegida con JWT firmado digitalmente.
- Trazabilidad: cada marcada de entrada o salida se registra de forma inmutable en la tabla asistencias. Las anomalías se registran automáticamente en la tabla alertas mediante el trigger de MySQL, sin posibilidad de omisión.
- Control de fraude: el sistema combina tres capas de verificación en cada marcada: PIN cifrado (identidad), GPS (ubicación física) y Device ID (dispositivo autorizado).
- Desacoplamiento: el frontend (React Native) y el backend (Node.js/Express) son componentes independientes que se comunican exclusivamente mediante la API REST, lo que permite actualizar cada parte sin afectar la otra.
- Consistencia automática: el cálculo de horas laboradas y la generación de alertas viven en un trigger de MySQL, garantizando que estos procesos ocurran siempre,

independientemente de si el registro se hace desde la app o directamente en la base de datos.

1.4 Diagrama de Proceso

El sistema tiene dos flujos principales: el flujo del empleado para marcar asistencia, y el flujo del administrador para gestionar el sistema desde el panel.



Flujo del empleado:

#	Paso	Descripción técnica
1	Abrir la app	PantallaInicio.tsx se muestra con dos opciones: 'Soy empleado' o 'Soy administrador'.
2	Seleccionar tipo de marcada	PantallaSeleccion.tsx muestra botones de Entrada y Salida. El empleado elige cuál corresponde.
3	Ingresar DNI y PIN	PantallaMarcar.tsx muestra un teclado numérico en pantalla. El empleado escribe su DNI de 13 dígitos y su PIN de 4 dígitos.
4	Captura de GPS y Device ID	La app solicita permiso de ubicación (expo-location) y captura la latitud/longitud actual. Captura también el Device ID del celular (expo-device + expo-application.getAndroidId()).
5	Envío al API	AsistenciaProvider llama a peticionPublica('POST', '/api/asistencias') enviando DNI, PIN, tipo, coordenadas y Device ID.
6	Validación en el backend	Routes/asistencias.ts: (1) busca el empleado por DNI, (2) verifica el PIN con bcrypt.compare(), (3) calcula distancia con Haversine vs. configuracion_gps, (4) verifica si el Device ID está en dispositivos_autorizados.
7	Inserción y trigger	Asistencia.create() inserta la fila. El TRIGGER after_insert_asistencia de MySQL se dispara solo: actualiza resumen_diario (horas laboradas/extras) y genera alertas automáticas si detecta anomalías.
8	Confirmación	PantallaConfirmacion.tsx muestra nombre del empleado, hora, fecha y estado ('Con alerta' o 'Válida') basado en la respuesta del backend.

Flujo del administrador:

#	Paso	Descripción técnica
1	Login con usuario y contraseña	PantallaLoginAdmin.tsx envía credenciales a POST /api/auth/login. El backend verifica con bcrypt.compare() y responde con un JWT firmado con expiración de 8 horas.
2	Acceso al panel	AdminProvider guarda el JWT en AsyncStorage. NavBar navega al PanelAdmin (Tab Navigator) con 4 pestañas: Empleados, Reportes, Alertas, Config.

3	Gestión de empleados	PantallaEmpleados lista empleados via GET /api/empleados. PantallaFormEmpleado permite crear (POST) o editar (PUT). El PIN nuevo se cifra con bcrypt antes de guardarse.
4	Consulta de reportes	PantallaReportes consume GET /api/reportes?fecha=YYYY-MM-DD. El backend ejecuta SELECT sobre la vista v_reporte_diario de MySQL y devuelve hora entrada, salida, horas laboradas y extras de cada empleado.
5	Revisión de alertas	PantallaAlertas lista alertas pendientes (GPS fuera del perímetro, dispositivo no registrado, dispositivo compartido). El admin puede marcarlas como revisadas con PUT /api/alertas/:id.
6	Configuración GPS	PantallaConfigGps permite editar nombre, latitud, longitud y radio del perímetro via GET y PUT /api/configuracion-gps. Incluye botón 'Usar mi ubicación actual' (expo-location).
7	Autorización de dispositivos	PantallaDispositivos muestra dispositivos pendientes (detectados en alertas) y autorizados. El admin puede autorizar un nuevo dispositivo (POST /api/dispositivos) o quitar uno existente (PUT /api/dispositivos/:id).
8	Cerrar sesión	El botón 'Salir' del header del PanelAdmin llama a cerrarSesion() del AdminProvider, que elimina el JWT de AsyncStorage y navega de regreso a PantallaInicio.

1.5 Requerimientos del sistema

Para el funcionamiento correcto del sistema SCAM se requiere:

- Dispositivo Android con Expo Go instalado (para desarrollo) o APK instalado directamente.
- Permisos de ubicación concedidos en el dispositivo móvil para la captura de GPS.
- Conexión a la misma red WiFi local donde corre el servidor backend.
- Servidor con Node.js v20.18.0 (instalado via NVM) y MySQL Server 9.7 corriendo en puerto 3306.
- Base de datos scam_danli creada en MySQL con las 9 tablas, 2 vistas y el trigger after_insert_asistencia.
- Archivo .env configurado con DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT y JWT_SECRET.

1.5.1 Vista de Casos de uso de la aplicación

Nombre del caso de uso	Actor	Descripción y ruta técnica
Inicio de sesión — Empleado	Empleado	El empleado escribe su DNI y PIN en PantallaMarcar.tsx. La app llama a POST /api/asistencias enviando las credenciales. El backend busca al empleado por DNI y verifica el PIN cifrado con bcrypt.compare(). Si coincide, registra la marcada.
Inicio de sesión — Administrador	Administrador	El administrador ingresa usuario y contraseña en PantallaLoginAdmin.tsx. La app llama a POST /api/auth/login. El backend verifica con bcrypt y responde con un JWT de 8 horas que el AdminProvider guarda en AsyncStorage.
Marcar asistencia con GPS	Empleado	Al presionar 'Marcar Entrada/Salida', la app solicita permiso de ubicación, captura coordenadas con expo-location y las envía al backend. El servidor calcula la distancia con Haversine y guarda dentro_perimetro=1 o 0. Si es 0, el trigger genera alerta 'gps_fuera_perimetro'.
Validación de dispositivo	Sistema (automático)	La app captura el Device ID con Device.modelName + Application.getAndroidId(). El backend busca ese Device ID en dispositivos_autorizados para ese empleado. Si no existe, guarda dispositivo_autorizado=0 y el trigger genera alerta 'dispositivo_no_registrado'.
Detección de dispositivo compartido	Sistema (automático)	El trigger comprueba si el mismo Device ID fue usado por otro empleado diferente en el mismo día. Si COUNT(DISTINCT id_empleado) > 0, genera alerta 'dispositivo_compartido' automáticamente.
Cálculo automático de horas	Sistema (trigger MySQL)	El trigger after_insert_asistencia calcula: horas_laboradas = TIMESTAMPDIFF(MINUTE, entrada, salida) / 60.0 y horas_extras = IF(horas_laboradas > horas_jornada, horas_laboradas - horas_jornada, 0). Actualiza la tabla resumen_diario.
Gestión de empleados	Administrador	Desde PantallaEmpleados (GET /api/empleados) el admin lista todos los empleados activos con su departamento. PantallaFormEmpleado permite crear (POST) o

		editar (PUT). El PIN nuevo siempre se cifra con <code>bcrypt.hash(pin, 10)</code> .
Consulta de reportes diarios	Administrador	PantallaReportes consume GET <code>/api/reportes?fecha=YYYY-MM-DD</code> . El backend ejecuta SELECT sobre la vista <code>v_reporte_diario</code> de MySQL, que combina <code>resumen_diario</code> + <code>empleados</code> + <code>departamentos</code> . Muestra hora entrada, hora salida, horas laboradas y horas extras de cada empleado.
Revisión de alertas	Administrador	PantallaAlertas lista alertas pendientes via GET <code>/api/alertas</code> . El admin puede marcar cada alerta como revisada con PUT <code>/api/alertas/:id</code> , registrando <code>revisada=1</code> , <code>revisada_por</code> (ID del admin) y <code>revisada_en</code> (fecha y hora).
Configuración del perímetro GPS	Administrador	PantallaConfigGps carga el perímetro actual con GET <code>/api/configuracion-gps</code> . El admin puede editar nombre, latitud, longitud y radio en metros, o usar el botón 'Usar mi ubicación actual' (<code>expo-location</code>). Guarda con PUT <code>/api/configuracion-gps/:id</code> .
Autorización de dispositivos	Administrador	PantallaDispositivos muestra alertas de 'dispositivo_no_registrado' pendientes. El admin puede autorizar el dispositivo con POST <code>/api/dispositivos</code> (vincula Device ID al empleado) o quitar un dispositivo existente con PUT <code>/api/dispositivos/:id</code> (<code>activo=0</code>).

2. Tecnología

2.1 Desarrollo

A continuación, se describen las tecnologías utilizadas en el desarrollo del sistema SCAM, todas apegadas a lo efectivamente implementado en el proyecto:

React Native + Expo	Framework para el desarrollo de la aplicación móvil (frontend). Permite escribir el código una sola vez en TypeScript/JavaScript y ejecutarlo en Android e iOS. Durante el desarrollo se usa Expo Go para probar en dispositivos físicos escaneando un código QR. La app tiene 11 pantallas organizadas en Stack Navigator y Tab Navigator mediante React Navigation.
TypeScript	Superset de JavaScript con tipado estático. Usado tanto en el frontend (React Native) como en el backend (Node.js). Permite detectar errores de

	tipo en tiempo de desarrollo (por ejemplo, pasar una cadena donde se espera un número), lo que mejora la confiabilidad del código.
Node.js v20.18.0 + Express	Entorno de ejecución y framework para el backend. Node.js permite ejecutar JavaScript en el servidor. Express es el framework que gestiona las rutas HTTP (GET, POST, PUT): recibe las peticiones de la app, aplica la lógica de negocio (validación de PIN, cálculo GPS, revisión de dispositivos) y responde en formato JSON. El servidor corre en el puerto 5000.
Sequelize ORM	Librería para Node.js que permite interactuar con MySQL mediante modelos de TypeScript (clases) en lugar de escribir SQL manualmente. El proyecto tiene 9 modelos (uno por tabla) definidos en src/Models/. Las relaciones (hasMany, belongsTo) se definen en Models/Index.ts para poder hacer consultas con JOIN usando la opción 'include'.
MySQL 9.7	Sistema de gestión de base de datos relacional. Almacena las 9 tablas del sistema. La característica más importante del diseño es el TRIGGER after_insert_asistencia, que se ejecuta automáticamente en cada marcada para calcular horas laboradas (TIMESTAMPDIFF), horas extras y generar alertas, todo sin que el backend lo solicite explícitamente.
JSON Web Token (JWT)	Estándar RFC 7519 para autenticación sin estado. Cuando el administrador inicia sesión correctamente, el backend genera un token firmado con una clave secreta (JWT_SECRET del archivo .env) con expiración de 8 horas. El frontend lo guarda en AsyncStorage y lo envía en la cabecera 'Authorization: Bearer <token>' en cada petición al panel admin. El middleware verificarToken.ts lo valida antes de dar acceso a las rutas protegidas.
bcryptjs	Librería para el cifrado seguro de PINs y contraseñas. Genera un hash unidireccional de 60 caracteres a partir del valor original. Nunca se puede obtener el PIN original a partir del hash; la verificación se hace comparando el texto plano con el hash usando bcrypt.compare(). El factor de costo usado es 10 (bcrypt.hash(pin, 10)).
expo-location	Plugin oficial de Expo para acceder al GPS del dispositivo. La función Location.requestForegroundPermissionsAsync() muestra el diálogo nativo de permisos. Location.getCurrentPositionAsync() obtiene las coordenadas actuales. Usado en PantallaMarcar.tsx y en PantallaConfigGps.tsx (para el botón 'Usar mi ubicación actual').
expo-device + expo-application	Plugins para identificar el dispositivo móvil. Device.modelName devuelve el nombre del modelo (ej: 'Redmi Note 12'). Application.getAndroidId() devuelve el Android ID único y persistente del dispositivo. Se combinan en un solo string: modelName + '_' + androidId, formando el Device ID que el sistema usa para el control anti-fraude.
AsyncStorage	Librería de almacenamiento persistente para React Native, análoga a localStorage en la web. El AdminProvider usa AsyncStorage.setItem('token', jwt) para guardar el JWT al iniciar sesión, y

	AsyncStorage.removeItem('token') al cerrar sesión. Sigue disponible incluso si el usuario cierra y vuelve a abrir la app.
Context API + Provider Pattern	Patrón de arquitectura del frontend. Cada dominio del sistema (sesión del admin, asistencias, empleados) tiene su propio Provider (AdminProvider, AsistenciaProvider, EmpleadoProvider) que guarda el estado usando useState y lo expone mediante createContext y un hook personalizado (useContextAdmin, useContextAsistencia, useContextEmpleado). Todas las pantallas acceden al estado global a través de estos hooks sin necesidad de pasar props manualmente.

2.2 Modelo Entidad Relación

